SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 3 **Duration : 1 Hour**
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I T NITHIN KUMAR(192424351) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: T NITHIN KUMAR

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift
   ii. D2 must work before D3 (Morning < Afternoon < Night)
   iii. Only one doctor per shift
   iv. D3 cannot work Morning
   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
c) Demonstrate how the robot applies AI concepts such as:

- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Question 1 – Backtracking Search

A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) subject to five constraints (i–v). Apply Backtracking Search with all intermediate steps and constraint checks, and state the final valid schedule.

Answer

1. Problem representation (CSP)

Item	Details
Variables	D1, D2, D3 (shift given to each doctor)
Domains	Each of D1, D2, D3 ∈ { Morning (M), Afternoon (A), Night (N) }, order M < A < N
Constraints	C1: D1 ≠ N C2: shift(D2) < shift(D3) C3: D1, D2, D3 all different (one doctor per shift) C4: D3 ≠ M C5: every doctor gets exactly one shift
Initial state	{ D1 = ?, D2 = ?, D3 = ? } – nothing assigned
Order used	Variables: D1 → D2 → D3. Values: Morning → Afternoon → Night

2. Step-by-step backtracking

Step	Assignment	Constraint Check	Result
1	D1 = Morning	C1: M ≠ N satisfied. C3 satisfied.	Accepted
2	D2 = Morning	C3: Morning already used by D1 – VIOLATED	Rejected
3	D2 = Afternoon	C3 satisfied. C2 pending (D3 not assigned).	Accepted
4	D3 = Morning	C4 VIOLATED (D3 ≠ M); C3 VIOLATED; C2 VIOLATED (A < M false)	Rejected → backtrack, next value
5	D3 = Afternoon	C3 VIOLATED (A used by D2); C2 VIOLATED (A < A false)	Rejected → backtrack, next value
6	D3 = Night	C1 ok (D1 = M). C2 ok (A < N). C3 ok (all different). C4 ok (N ≠ M). C5 ok.	Accepted – complete valid assignment

3. Final valid schedule

Shift	Doctor
Morning	D1
Afternoon	D2
Night	D3

FINAL ANSWER

D1 → Morning, D2 → Afternoon, D3 → Night

4. Verification

C1: D1 works Morning, not Night. C2: D2 (Afternoon) works before D3 (Night). C3: Morning, Afternoon and Night each have exactly one doctor. C4: D3 works Night, not Morning. C5: D1, D2, D3 each have exactly one shift. Hence all five constraints are satisfied.

Question 2 – Grid Search

A robot operates in a 5×5 grid. It must move from Start (S) to Goal (G). Obstacles block certain paths. Constraints: (i) Up, Down, Left, Right; (ii) cost of each move = 1; (iii) cannot pass through obstacles; (iv) use Manhattan Distance heuristic. Using the above constraints and BFS, show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

Answer

Assumption

The question gives only the grid size (5×5) and not the positions of S, G and obstacles. The following example grid is **assumed** to demonstrate the method: S = (1,1), G = (5,5), obstacles = (1,4), (2,2), (2,4), (3,2), (4,3), (4,4). Cells are written as (row, column).

	1	2	3	4	5
1	S			#	
2		#		#	
3		#			
4			#	#	
5					G

1. Search strategy

Ordinary BFS uses a FIFO queue and no heuristic. Since the question requires a **Manhattan Distance heuristic and a priority queue**, the BFS-style search is performed as **A* search**: the queue is ordered by f(n) = g(n) + h(n). All moves cost 1, so the result is also the shortest (BFS) path.

Term	Meaning
g(n)	Actual cost from Start to node n (moves made so far)
h(n)	Manhattan Distance to Goal: $h(n) = \|x_1 - x_2\| + \|y_1 - y_2\|$ → h(Start) = |1−5| + |1−5| = 8
f(n)	f(n) = g(n) + h(n): estimated total cost through n → f(Start) = 0 + 8 = 8

Tie-breaking: smaller f first, then smaller h, then the node inserted earlier. Neighbours are checked in the order Up, Down, Left, Right.

2. Node expansion and priority queue updates

Entry format: (row,col)[g, h, f]. Initial queue: { (1,1)[0, 8, 8] }

Step	Current Node	g(n)	h(n)	f(n)	Neighbours Added	Priority Queue
1	(1,1)	0	8	8	(2,1) [g=1, h=7, f=8] (1,2) [g=1, h=7, f=8]	(2,1)[1, 7, 8], (1,2)[1, 7, 8]
2	(2,1)	1	7	8	(3,1) [g=2, h=6, f=8] Skipped: (1,1) closed, (2,2) obstacle	(3,1)[2, 6, 8], (1,2)[1, 7, 8]

Step	Current Node	g(n)	h(n)	f(n)	Neighbours Added	Priority Queue
3	(3,1)	2	6	8	(4,1) [g=3, h=5, f=8] Skipped: (2,1) closed, (3,2) obstacle	(4,1)[3, 5, 8], (1,2)[1, 7, 8]
4	(4,1)	3	5	8	(5,1) [g=4, h=4, f=8] (4,2) [g=4, h=4, f=8] Skipped: (3,1) closed	(5,1)[4, 4, 8], (4,2)[4, 4, 8], (1,2)[1, 7, 8]
5	(5,1)	4	4	8	(5,2) [g=5, h=3, f=8] Skipped: (4,1) closed	(5,2)[5, 3, 8], (4,2)[4, 4, 8], (1,2)[1, 7, 8]
6	(5,2)	5	3	8	(5,3) [g=6, h=2, f=8] Skipped: (5,1) closed	(5,3)[6, 2, 8], (4,2)[4, 4, 8], (1,2)[1, 7, 8]
7	(5,3)	6	2	8	(5,4) [g=7, h=1, f=8] Skipped: (4,3) obstacle, (5,2) closed	(5,4)[7, 1, 8], (4,2)[4, 4, 8], (1,2)[1, 7, 8]
8	(5,4)	7	1	8	(5,5) [g=8, h=0, f=8] Skipped: (4,4) obstacle, (5,3) closed	(5,5)[8, 0, 8], (4,2)[4, 4, 8], (1,2)[1, 7, 8]
9	(5,5)	8	0	8	– (goal reached)	Goal removed → STOP. Left: (4,2)[4,4,8], (1,2)[1,7,8]

At step 4, (5,1) and (4,2) have equal f and h, so (5,1) (inserted first) is taken. Node (1,2) stays in the queue but is never expanded because nodes with smaller h are preferred.

3. Optimal path and cost

OPTIMAL PATH

(1,1) → (2,1) → (3,1) → (4,1) → (5,1) → (5,2) → (5,3) → (5,4) → (5,5)
(Down ×4, then Right ×4)

TOTAL COST

Number of moves × 1 = 8 × 1 = **8**

4. Why the path is optimal

- The Manhattan Distance from S to G is 8, so no path can have fewer than 8 moves; the path found has exactly 8 moves.
- h(n) never over-estimates the real cost (admissible and consistent), so the first time A* removes the Goal from the queue, its cost is the minimum.
- BFS gives the same cost 8 but expands 18 nodes; A* expands only 9. No obstacle is crossed and every move is Up/Down/Left/Right with cost 1.

5. Role of the Manhattan heuristic

It estimates how far each node is from the goal, so nodes nearer to the goal are expanded first. This directs the search towards G and reduces the number of expanded nodes without losing optimality.

Question 3 – Autonomous Rescue Robot

An autonomous rescue robot in a disaster-affected building (partially observable grid of rooms) must go from S to a survivor at G. Constraints: moves up/down/left/right; blocked cells cannot be traversed; it can observe only adjacent cells; each move costs 1 and entering a risky zone costs an additional 2; minimize total cost with safe navigation; update knowledge dynamically (online search). (a) Analyze the constraints. (b) Develop a solution approach. (c) Apply intelligent agent, rationality and environment type. (d) Justify the approach.

(a) Analysis of constraints

Constraint	Meaning	Effect on Decision Making
Moves: up, down, left, right	At most 4 neighbouring cells; no diagonal moves	Limits the actions in each state to 4
Blocked cells	Cannot be entered	Removed from the action list; if the plan uses one, the robot must replan
Observe adjacent cells only	Partial observability	Robot cannot plan on the true map; it plans on current knowledge and corrects it
Move cost = 1	Every step costs 1	Shorter paths are preferred
Risky zone: +2	Entering a risky cell costs 1 + 2 = 3	Used only if the detour costs more than 2 extra; balances speed and safety
Minimize cost, safe navigation	Lowest cost, never enter blocked cells, avoid risk	Choose the cheapest path where risk is turned into cost
Dynamic knowledge update (online)	Map is learnt while moving	Update the map after each move and replan when needed

Priority: blocked cells (hard constraint) → risky zones (soft, handled by +2 cost) → total cost (objective) → sensing/updating (decides when to replan).

(b) Solution approach: Online Search Agent using Uniform Cost Search (UCS)

UCS expands the node with the lowest path cost g(n), so it works with unequal costs (1 and 3). Because the map is unknown, the robot repeats: **sense → update knowledge → plan with UCS → take one step**. Assumptions: the sensor shows whether each adjacent cell is free, risky or blocked; unseen cells are assumed free (cost 1).

1. **Initial state:** robot at S; only S, G and grid size are known.
2. **Observations:** type of the adjacent cells (free / risky / blocked).
3. **Action selection:** run UCS on the known map; execute only the first move.
4. **Cost calculation:** step cost = 1 (normal cell) or 1 + 2 = 3 (risky cell); path cost g(n) = sum of step costs.
5. **Obstacles:** observed blocked cells are stored and never used in later plans.
6. **Risky zones:** stored with cost 3; UCS avoids them if a cheaper path exists.
7. **Updating knowledge:** after every move, write the new observations into the map.
8. **Replanning:** if a plan becomes blocked or costlier, run UCS again from the current cell.
9. **Reaching the survivor:** repeat until the robot enters G.
10. **Minimizing total cost:** every step follows the cheapest plan on the latest knowledge.

```
pos = S;  total = 0;  known = empty map
while pos != G:
    observe adjacent cells; update known (blocked / risky / free)
    path = UCS(known, pos, G)        // cost(cell) = 1, or 3 if risky
    next = first cell after pos in path
    move to next;  total = total + cost(next);  pos = next
return total
```

Example (assumed 4×4 map, unknown to the robot): S = (1,1), G = (4,4), blocked = (1,2), (2,2), risky = (3,2).

Step	Robot at	Observed	UCS plan (estimated cost)	Action	Cost	Total
1	(1,1)	(1,2) blocked	(2,1)→(2,2)→(2,3)→(2,4)→(3,4)→(4,4) = 6	Down to (2,1)	1	1
2	(2,1)	(2,2) blocked	Replan: (3,1)→(3,2)→(3,3)→(3,4)→(4,4) = 5	Down to (3,1)	1	2
3	(3,1)	(3,2) risky	Via (3,2) = 3+1+1+1 = 6; via (4,1)→(4,2)→(4,3)→(4,4) = 4 → choose 4	Down to (4,1)	1	3
4	(4,1)	all free	(4,2)→(4,3)→(4,4) = 3	Right to (4,2)	1	4
5	(4,2)	(4,3) free	(4,3)→(4,4) = 2	Right to (4,3)	1	5
6	(4,3)	(4,4) survivor	(4,4) = 1	Right to (4,4)	1	6

RESULT

Path: (1,1) → (2,1) → (3,1) → (4,1) → (4,2) → (4,3) → (4,4). **Total cost = 6**; no blocked or risky cell entered (using the risky cell would have cost 8).

(c) Application of AI concepts

Concept	Application to the rescue robot
Intelligent agent	The robot perceives its surroundings through sensors (adjacent cells, position) and acts through actuators (moves). It is a goal-based, utility-based agent: the goal is to reach the survivor and path cost measures how good a choice is. Agent function: percept → update knowledge → UCS → one move.
Rationality	At every step it chooses the action with the lowest expected total cost on its current knowledge; never enters known blocked cells; adds +2 to risky cells so risk is accepted only when the detour costs more; uses every new observation to improve its plan.

Environment property	Classification	Reason
Observability	Partially observable	Only adjacent cells are seen
Agents	Single agent	Only one robot acts
Determinism	Deterministic (assumed)	Each move reaches the intended cell; with slipping it would be stochastic
Episodic / sequential	Sequential	Each move affects later positions, options and total cost
Static / dynamic	Dynamic (knowledge changes)	A disaster site can change; the robot keeps updating its knowledge and replans
Discrete / continuous	Discrete	Grid cells and four finite actions
Known / unknown	Unknown (partially known)	Rules and costs are known; blocked and risky cells are learnt by exploration

(d) Justification and comparison

Requirement	How UCS-based Online Search Agent handles it
Different movement costs	Orders nodes by real cost g(n) (1 and 3) and returns the cheapest path
Risky zones	Risk is added as cost (+2); avoided when a cheaper path exists
Obstacles	Blocked cells are excluded from every plan
Limited information	Plans on the known map and corrects it each step (unknown cells assumed free)
Dynamic knowledge	Replans after every knowledge update
Safe navigation	Never enters blocked cells; penalizes risky cells

Method	Limitation for this problem
BFS	Finds the fewest moves, ignores cost; may go through a risky cell (cost 3)
DFS	Not optimal; may take a long or risky route
Greedy Best-First	Uses only h(n); ignores cost paid and risk; not optimal
A*	Also suitable ($f = g + h$, Manhattan h is admissible since every step costs ≥ 1); gives the same paths as UCS with fewer expansions but needs a heuristic
UCS in an online agent	Handles costs exactly, needs no heuristic, replans with new information – chosen

CONCLUSION

The rescue robot is a rational goal-based agent in a partially observable, sequential, discrete and partially unknown environment. Repeating sense → update → UCS → move, it avoids blocked cells, pays for risky cells only when necessary and reaches the survivor at minimum known cost (6 in the example).